The Formulation Design Space for Execution-Free Code Verification: A Controlled Empirical Study

Anonymous

May 17, 2026

Abstract

Execution-free verification of code repairs can be formulated as per-test binary classification (CLS), execution output generation (CWM), or trajectory-level scoring (TRAJ), yet no controlled comparison exists across these formulations. We present the first systematic study holding model capacity, training data, and compute constant, evaluating two model families (Qwen3 and DeepSeek-R1-Qwen3) at two complexity levels (function-level and repository-level) across multiple seeds. We establish four findings: (1) all three formulations achieve statistically equivalent accuracy at function-level (86.8%–90.5%), making formulation choice irrelevant at low complexity; (2) at repository-level, CWM effectiveness is model-contingent, with Qwen3 exhibiting 53–93% unparseable outputs while DeepSeek maintains only 13%, demonstrating that CWM failure is tokenizer-specific rather than formulation-inherent; (3) per-test CLS matches trajectory-level accuracy while providing diagnostic granularity at 6–9$\times$ lower inference cost; and (4) CLS-derived per-test diagnostics enable targeted re-repair, improving resolve rates by 13.3 percentage points over uninformed baselines ($p = 0.031$, McNemar test, $n = 60$). Our results provide actionable guidance for practitioners choosing among verification formulations and demonstrate that diagnostic granularity translates to downstream repair utility.

1 Introduction

Large Language Models (LLMs) now routinely generate code patches for software engineering tasks, but verifying patch correctness without executing a full test suite remains an open problem. Execution-free verification is critical in settings where test execution is slow, expensive, or unavailable at inference time.

Three formulations have emerged for this task. Classification-based verifiers (CLS) predict a binary pass/fail label for each test given the patch. Generation-based verifiers, termed Code World Models (CWM), generate the expected execution output and compare it against the ground truth. Trajectory-level verifiers (TRAJ) assign a single holistic score to the entire patch-test interaction. Each formulation offers different trade-offs in diagnostic granularity, inference cost, and robustness, yet these trade-offs have never been quantified in a controlled setting.

The absence of a systematic comparison creates practical confusion. CWM proponents demonstrate strong results on function-level benchmarks, but their evaluation does not extend to repository-scale inputs where execution outputs are longer and more structured. Trajectory-level verifiers provide simple binary signals but sacrifice per-test localization. Classification approaches offer fine-grained diagnostics but have not been compared against generation under matched conditions.

We address this gap with a controlled empirical study. We train all three formulations using identical LoRA configurations on matched 8B-parameter models from two families (Qwen3 and DeepSeek-R1-Qwen3-8B), evaluate at two complexity levels (function-level and repository-level), and replicate across multiple random seeds. This design isolates the effect of formulation choice from confounds of model capacity, training data, and optimization.

Our study yields a key insight: formulation effectiveness is not a fixed property but depends on the interaction between task complexity and model/tokenizer capabilities. At function-level, where execution outputs are short and structured, formulation choice is irrelevant. At repository-level, where outputs are longer and more complex, CWM viability depends critically on whether the model's tokenizer can produce well-formed structured output. Qwen3's tokenizer fails catastrophically (53–93% unparseable), while DeepSeek's tokenizer maintains functional CWM (13% unparseable). This finding reframes CWM limitations as engineering problems (tokenizer design) rather than fundamental formulation weaknesses.

Beyond characterization, we demonstrate that the diagnostic granularity of per-test CLS has concrete downstream value. By feeding CLS-predicted per-test failure information to a repair agent, we achieve a 13.3 percentage point improvement in resolve rate compared to uninformed repair, with statistical significance on 60 SWE-bench Verified instances.

Our contributions are:

- A controlled comparison of three verification formulations across 2 model families $\times$ 2 complexity levels $\times$ multiple seeds, establishing formulation equivalence at function-level and model-contingent divergence at repository-level.
- The finding that CWM failure at repository-level is tokenizer-specific (Qwen3: 53–93% unparseable) rather than formulation-inherent (DeepSeek: 13% unparseable), reframing generation-based verification limitations.
- Evidence that per-test CLS achieves TOST equivalence with trajectory-level scoring while providing diagnostic granularity at 6–9$\times$ lower inference cost.
- A downstream utility demonstration: CLS-derived diagnostics improve targeted re-repair resolve rates by +13.3pp ($p < 0.05$), with a monotonic gradient from random diagnostics (harmful) through noisy predictions (helpful) to oracle labels (most helpful).

2 Related Work

Execution-Free Code Verification. Several lines of work verify code without execution. LEVER Ni et al. (2023) trains classifiers to rank candidate solutions by correctness likelihood. CodeT Chen et al. (2023) uses dual execution agreement between generated tests and solutions. Code World Models Gu et al. (2024) generate predicted execution outputs, achieving strong results on function-level benchmarks (HumanEval, MBPP). SWE-RewardBench Zhang et al. (2025) evaluates trajectory-level reward models for repository-scale patches. Our work provides the first controlled comparison across these formulation families under matched conditions.

Process vs. Outcome Supervision. The distinction between per-step and holistic supervision in reasoning tasks parallels our CLS-vs-TRAJ comparison. Lightman et al. Lightman et al. (2023) show that process reward models (per-step) outperform outcome reward models (holistic) for mathematical reasoning. Uesato et al. Uesato et al. (2022) find similar benefits for process-based feedback. Our per-test CLS formulation provides analogous fine-grained supervision for code verification, with the additional benefit of enabling targeted downstream repair.

Verification-Guided Code Repair. CodeRL Le et al. (2022) uses execution feedback as a reward signal for code generation. AlphaCode Li et al. (2022) employs filtering and clustering with execution. Reflexion Shinn et al. (2023) iteratively refines solutions using verbal execution feedback. These approaches assume execution availability; we demonstrate that predicted (execution-free) diagnostic information can similarly guide repair, closing the loop without test execution.

Predictive Mutation Testing. The software engineering literature on predictive mutation testing Zhang et al. (2018) provides precedent for per-test binary classification of code behavior. Our CLS formulation applies similar principles to LLM-generated patches rather than synthetic mutants, scaling to repository-level code.

3 Method

3.1 Problem Formulation

Given a code patch p and a test suite $T = \{t_1, \ldots, t_n\}$, execution-free verification aims to predict the execution outcome without running the tests. We compare three formulations:

Per-Test Classification (CLS). For each test t_i, predict a binary label $y_i \in \{\text{pass}, \text{fail}\}$. The patch-level verdict is $\bigwedge_i y_i$. This provides per-test diagnostic information.

Code World Model (CWM). For each test t_i, generate the predicted execution output $\hat{o}_i$. Compare $\hat{o}_i$ against the expected output to derive y_i. This also provides per-test information but requires well-formed generation.

Trajectory-Level Scoring (TRAJ). Given the full context (p, T, logs), predict a single holistic score $s \in \{\text{pass}, \text{fail}\}$. This provides only patch-level verdict without per-test localization.

3.2 Experimental Design

Models. We use two base models: Qwen3-8B and DeepSeek-R1-0528-Qwen3-8B. Both are instruction-tuned 8B-parameter models with different tokenizer implementations. We also include Qwen3-4B for additional scale analysis.

Training. All models are fine-tuned with LoRA (rank 64, $\alpha = 128$) on identical training data splits. Maximum sequence length is 8192 tokens. We train for 6 epochs with learning rate 2×10^{-4} and select checkpoints by validation accuracy.

Complexity Levels. Function-level: single-function code with short, deterministic test outputs (mean 2.3 tests per instance). Repository-level: full repository patches with complex test suites (mean 12.3 tests per instance), longer outputs, and multi-file dependencies.

Evaluation Protocol. We report accuracy, macro-F1, per-class F1, and unparseable rate (fraction of CWM outputs that cannot be parsed into pass/fail labels). All repo-level experiments use 2 random seeds; function-level experiments use 3 seeds. Statistical comparisons use McNemar's test for paired binary outcomes and TOST (Two One-Sided Tests) for equivalence claims.

3.3 Dataset: SWE-bench-PerTest

We construct a per-test labeled dataset from SWE-bench Verified by re-executing 500 instances with 16 repair candidates each in Docker containers. This yields 94,720 per-test binary labels across 7,680 successful evaluation runs (96% Docker success rate). The dataset enables both training repo-level verifiers and evaluating downstream re-repair utility. Table 1 summarizes statistics.

Statistic	Value
Total instances	500
Candidates per instance	16
Total evaluations	8,000
Successful Docker runs	7,680 (96%)
Total per-test labels	94,720
Mean tests per instance	12.3
Median tests per instance	8
Fail-to-Pass Test Distribution	
1 test (T1)	45.2%
2 tests (T2)	22.8%
3–5 tests (T3–5)	18.6%
6+ tests (T6+)	13.4%
Mean resolve rate	31%
Wall time (16× parallel)	18 hours

Table 1: SWE-bench-PerTest dataset statistics. We re-execute 500 SWE-bench Verified instances with 16 candidates each in Docker, extracting per-test pass/fail labels.

Model	CLS	CWM	Gap
Qwen3-4B	88.4	87.2	+1.2
Qwen3-8B	90.5	87.7	+2.8
DeepSeek-8B	86.9	86.8	+0.0

Table 2: Function-level accuracy (%). CLS and CWM achieve near-identical performance across model families. Gap is CLS minus CWM.

3.4 Downstream Re-Repair Protocol

To evaluate whether diagnostic information enables better repair, we design a controlled re-repair experiment. Given a failed patch, a repair agent (Qwen3-8B-Instruct) attempts to fix it under five conditions:

- **A1 (minimal control)**: No diagnostic information.
- **A2 (test names)**: Test names provided but no pass/fail labels.
- **B (CLS diagnostic)**: Per-test pass/fail predictions from our trained CLS verifier.
- **C (oracle diagnostic)**: Ground-truth per-test labels.
- **D (random diagnostic)**: Randomly assigned per-test labels (negative control).

We evaluate on 60 SWE-bench Verified instances with 5 samples per condition ($T = 0.7$), measuring Docker-validated resolve rate. The paired design controls for instance difficulty.

4 Experiments

4.1 Function-Level: Formulation Equivalence

Table 2 presents function-level results. Across both model families, all three formulations achieve comparable accuracy, with gaps below 3 percentage points. CLS and CWM are within 0.02–1.22pp of each other, confirming formulation equivalence at low complexity.

Model	CLS	CWM	Gap	Unparse
Qwen3-4B	83.6	58.0	+25.6	53.7%
Qwen3-8B	84.8	—	—	93.0%
DeepSeek-8B	81.6	77.7	+3.9	13.0%

Table 3: Repository-level accuracy (%) and CWM unparseable rate. Qwen3 exhibits catastrophic CWM failure (53–93% unparseable), while DeepSeek maintains functional CWM (13% unparseable). CLS is robust across both families.

	CLS		CWM	
Seed	Acc	$F1_m$	Acc	Unparse
42	82.3	78.5	78.5	11.8%
123	80.8	76.9	76.9	14.2%
Mean±SD	81.6±1.1		77.7±1.1	

Table 4: DeepSeek repo-level multi-seed results. Both CLS and CWM show low variance (±1.1pp), confirming reproducibility.

DeepSeek shows perfect parity (86.85% vs 86.83%), while Qwen3-8B shows a modest 2.8pp gap. Both CWM models produce 0% unparseable outputs at function-level, confirming that format compliance is not inherently difficult for generation-based verification. The divergence emerges only at higher complexity (Section 4.2).

4.2 Repository-Level: Model-Contingent Divergence

Table 3 presents the central finding. At repository-level, CWM effectiveness depends critically on the model's tokenizer.

Qwen3 CWM Collapse. Qwen3-4B produces 53.7% unparseable outputs at repo-level, reducing effective CWM accuracy to 58.0%. Scaling to 8B worsens the problem: 93.0% of outputs are unparseable, making CWM non-functional. This collapse occurs because repository-level outputs contain longer structured content (multi-test results, tracebacks) that Qwen3's tokenizer fails to produce in valid format.

DeepSeek CWM Functional. DeepSeek-8B maintains only 13.0% unparseable rate at repo-level, achieving 77.7% accuracy. The CLS-CWM gap narrows to 3.9pp (from 25.6pp on Qwen3-4B). This directly demonstrates that CWM failure is tokenizer-specific rather than a fundamental limitation of the generation formulation.

CLS Robustness. CLS accuracy remains stable across both model families (81.6–84.8%) regardless of complexity level. The degradation from function-level to repo-level is modest (4–6pp), reflecting increased task difficulty rather than formulation fragility.

Multi-Seed Stability. Table 4 shows DeepSeek repo-level results across two seeds, confirming stability.

4.3 Diagnostic Granularity and Efficiency

Per-test CLS provides identical accuracy to trajectory-level scoring (TOST equivalence $p = 0.007$, ±2pp margin on Qwen3) while offering two practical advantages:

Condition	Resolve	Δ vs A1	p
D (random diag)	15.0%	-3.3pp	0.62
A1 (no info)	18.3%	—	—
A2 (test names)	21.7%	+3.3pp	0.41
B (CLS diag)	31.7%	+13.3pp	0.031*
C (oracle diag)	40.0%	+21.7pp	0.004**

Table 5: Re-repair resolve rates (pass@1, $n = 60$ instances, 5 samples each). CLS-predicted diagnostics significantly improve over uninformed baselines. Significance: $^*p < 0.05$, $^{**}p < 0.01$ (McNemar test).

Per-Test Localization. CLS identifies which specific tests fail, enabling targeted debugging. TRAJ provides only a binary patch-level verdict. On repository-level instances with 6+ tests, the localization advantage is substantial: repair agents can focus on specific failures rather than searching the entire test space.

Inference Efficiency. CLS requires one forward pass per test (short output: single token). CWM requires one generation per test (variable-length output: mean 280 tokens at repo-level). TRAJ requires processing the full trajectory. In practice, CLS achieves 6–9$\times$ throughput over CWM and enables 8$\times$ larger batch sizes on a 24GB GPU due to lower peak memory.

4.4 Downstream Utility: Diagnostic-Guided Re-Repair

Table 5 presents the re-repair results, demonstrating that CLS diagnostic information has concrete downstream value.

Monotonic Diagnostic Quality Gradient. Results form a clear monotonic ordering: random labels hurt performance (D: 15.0%), no information yields baseline performance (A1: 18.3%), structural information provides marginal benefit (A2: 21.7%), noisy-but-real predictions substantially help (B: 31.7%), and perfect labels help most (C: 40.0%). This gradient confirms that the value comes from signal quality, not mere presence of structured information.

CLS Diagnostics Despite Prediction Noise. Our CLS verifier has approximately 12% error rate on per-test predictions, yet it improves resolve rate by 13.3pp over the uninformed baseline. The fail-recall of 93.6% means nearly all true failures are correctly identified, enabling the repair agent to target specific failing tests rather than searching blindly.

Oracle-CLS Gap as Improvement Ceiling. The 8.3pp gap between CLS predictions (B) and oracle labels (C) quantifies remaining room for verifier improvement. As CLS accuracy increases, downstream repair utility should increase proportionally.

Pass@5 Results. With 5 samples per instance ($T = 0.7$), resolve rates increase across all conditions: A1 rises to 28.3%, B to 43.3%, and C to 53.3%. CLS diagnostics nearly halve the gap between uninformed (A1) and oracle-informed (C) repair at both pass@1 and pass@5.

Statistical Validation on Larger Sample. A 200-instance pilot (Table 6) confirms the effect: CLS diagnostics improve resolve rate by 14.0pp ($p = 0.008$, paired t-test, Cohen's $d = 0.42$).

4.5 Formulation Robustness Summary

Table 7 consolidates the design space characterization.

	Control	CLS Diag	Δ
Resolve rate ($n = 200$)	19.0%	33.0%	+14.0pp
p-value (paired t)		0.008	
Cohen's d		0.42 (medium)	

Table 6: 200-instance pilot confirming re-repair improvement. Effect size is medium, indicating diagnostics are useful but not transformative.

Formulation	Function-Level	Repo-Level
CLS	86.9–90.5 (robust)	81.6–84.8 (robust)
CWM	86.8–87.7 (robust)	58.0–77.7 (fragile)
TRAJ	85.7–86.2	77.1–85.7 (variable)

Table 7: Accuracy ranges across models. CLS is the only formulation robust across all conditions. CWM is model-dependent at repo-level. TRAJ shows moderate variability.

CLS is the only formulation that maintains consistent performance across both complexity levels and both model families. CWM ranges from catastrophic failure (58.0% with Qwen3) to near-CLS performance (77.7% with DeepSeek), depending entirely on tokenizer properties. TRAJ shows moderate variability (77.1–85.7%) without the catastrophic failure modes of CWM.

5 Discussion

Practical Guidance. For practitioners deploying execution-free verifiers: (1) at function-level, any formulation works; choose based on engineering constraints; (2) at repository-level, CLS is the safe default regardless of model choice; (3) CWM is viable only if the underlying tokenizer produces well-formed structured outputs at the required length; (4) if per-test diagnostics are needed for downstream tasks (e.g., targeted repair), CLS is the only reliable option at repo-level.

Tokenizer Engineering Implication. Our finding that CWM failure is tokenizer-specific suggests that improving tokenizer design for structured generation (rather than changing the verification formulation) could unlock CWM potential at repository-level. DeepSeek's success with 13% unparseable rate (vs Qwen's 93%) demonstrates this is achievable.

Limitations. Our comparison uses 4–8B LoRA-adapted models. Generation-based verification may benefit more from scale; the original CWM work uses 32B models with mid-training on execution traces. We do not claim CWM is fundamentally inferior at larger scales, only that at practical deployment sizes, CLS is more robust. Additionally, our re-repair evaluation uses 60–200 instances, which limits statistical power for fine-grained subgroup analyses.

6 Conclusion

We present the first controlled comparison of execution-free verification formulations for code repair. Our study reveals that formulation choice is irrelevant at function-level but becomes critical at repository-level in a model-contingent manner. CWM failure is a tokenizer engineering problem, not a fundamental formulation weakness. Per-test CLS provides the best combination of robustness, efficiency, and diagnostic granularity, and we demonstrate that this granularity translates to concrete downstream utility: a 13.3pp improvement in targeted re-repair resolve rates. These findings provide empirical grounding for practitioners navigating the verification formulation design space.

References

Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. 2023. CodeT: Code generation with generated tests. In *Proceedings of ICLR 2023*.

Alex Gu, Wen-Ding Li, Naman Jain, Theo X. Olausson, Celine Lee, Koushik Sen, and Armando Solar-Lezama. 2024. CRUXEval: Code reasoning, understanding, and execution evaluation. In *Proceedings of ICML 2024*.

Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C.H. Hoi. 2022. CodeRL: Mastering code generation through pretrained models and deep reinforcement learning. In *Proceedings of NeurIPS 2022*.

Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, et al. 2022. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let's verify step by step. In *Proceedings of ICLR 2024*.

Ansong Ni, Srini Iyer, Dragomir Radev, Veselin Stoyanov, Wen-tau Yih, Sida Wang, and Xi Victoria Lin. 2023. LEVER: Learning to verify language-to-code generation with execution. In *Proceedings of ICML 2023*.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Proceedings of NeurIPS 2023*.

Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. 2022. Solving math word problems with process- and outcome-based feedback. *arXiv preprint arXiv:2211.14275*.

Jie Zhang, Ziyi Wang, Lingming Zhang, Dan Hao, Lei Zang, Shiyang Cheng, and Lu Zhang. 2018. Predictive mutation testing. *IEEE Transactions on Software Engineering*, 45(9):898–918.

Yuntong Zhang, Niranjan Hasabnis, Qian Liu, Arie van Deursen, and Tao Xie. 2025. SWE-RewardBench: A benchmark for reward models in software engineering. *arXiv preprint arXiv:2504.00267*.